

Guía 03 - Creación de una Web API REST con ASP.NET Core (.NET 8)

1. TÍTULO: Guía 03 - Creación de una Web API REST con ASP.NET Core (.NET 8)

2. DATOS INFORMATIVOS

- **Asignatura** : Arquitectura Orientada a Servicios
- **Semestre Académico** : 2026-2
- **Ciclo** : X
- **Categoría** : Estudios Electivos de Especialidad (EEE)
- **Docente** : Mtro. Ing. Luis Alvaro Monzón Condori
- **Duración** : 04 horas

3. COMPETENCIAS A SEGUIR.

Al finalizar la guía, el estudiante será capaz de:

- Comprender el funcionamiento de una Web API REST dentro de una arquitectura distribuida.
- Crear y configurar un proyecto ASP.NET Core Web API en .NET 8.
- Implementar **endpoints** utilizando métodos HTTP (GET, POST, PUT, DELETE).
- Probar servicios mediante Swagger.
- Consumir una API REST desde una aplicación de consola usando **HttpClient**.

4. REQUISITOS.

- Haber desarrollado la Guía 02 - Configuración del entorno y primeros pasos con .NET 8.
- Visual Studio 2022 o superior con soporte para .NET 8.
- SDK de .NET 8 instalado.
- Conocimientos básicos de programación en C#.
- Comprensión básica de arquitectura cliente-servidor

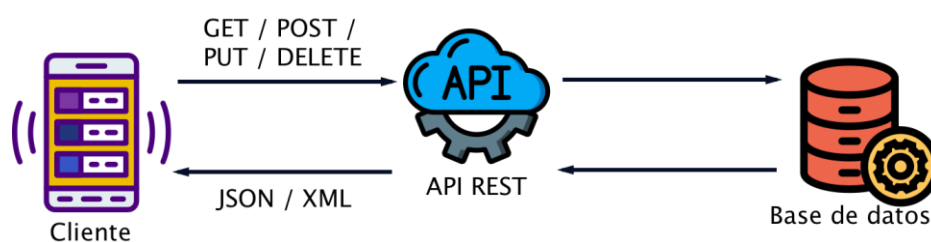
5. BASES TEÓRICAS.

5.1. ¿Qué es una Web API REST?

Una **Web API REST** es un **servicio** que permite la **comunicación entre aplicaciones** mediante el protocolo **HTTP**. Se basa en el intercambio de datos generalmente en formato **JSON** y permite desacoplar módulos dentro de un sistema distribuido.

En arquitecturas modernas:

- Cada módulo puede exponer funcionalidades como servicios.
- Las aplicaciones consumidoras acceden a esos servicios mediante solicitudes HTTP.
- La comunicación sigue el modelo Cliente - Servidor.



Funcionamiento de una Web API REST

Fuente: <https://dossetenta.com/que-es-una-api-rest/>

5.2. Métodos HTTP en servicios REST.

Método	Función
GET	Obtener información
POST	Crear nuevos recursos
PUT	Actualizar recursos existentes
DELETE	Eliminar recursos

5.3. ASP.NET Core Web API

ASP.NET Core es el framework de .NET diseñado para crear aplicaciones web y APIs.

Componentes principales en una Web API:

Program.cs	→ Punto de inicio de la aplicación.
Controllers	→ Contienen los endpoints.
Models	→ Representan las entidades del sistema.
Swagger	→ Herramienta para probar servicios.

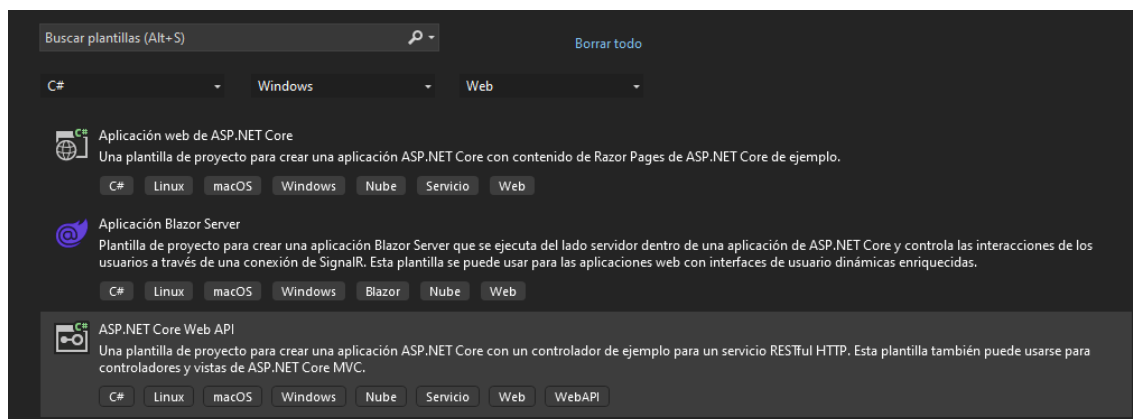
En esta guía se construirá una API básica que permitirá gestionar estudiantes como simulación de un módulo académico.

6. DESARROLLO DE LA GUÍA:

6.1. ACTIVIDAD 01: Creación del proyecto Web API.

Objetivo: Crear el proyecto base y comprender su estructura.

1. Abrir Visual Studio.
2. Crear un proyecto **ASP.NET Core Web API**.



3. Nombre del proyecto: **ServicioAcademicoAPI**.
4. Seleccionar Framework **.NET** más actualizado.
5. Marcar la opción **Habilitar Compatibilidad con OpenAPI (Swagger)**.

Información adicional

ASP.NET Core Web API C# Linux macOS Windows Nube Servicio Web WebAPI

Framework ⓘ
.NET 7.0 (Sin soporte técnico)

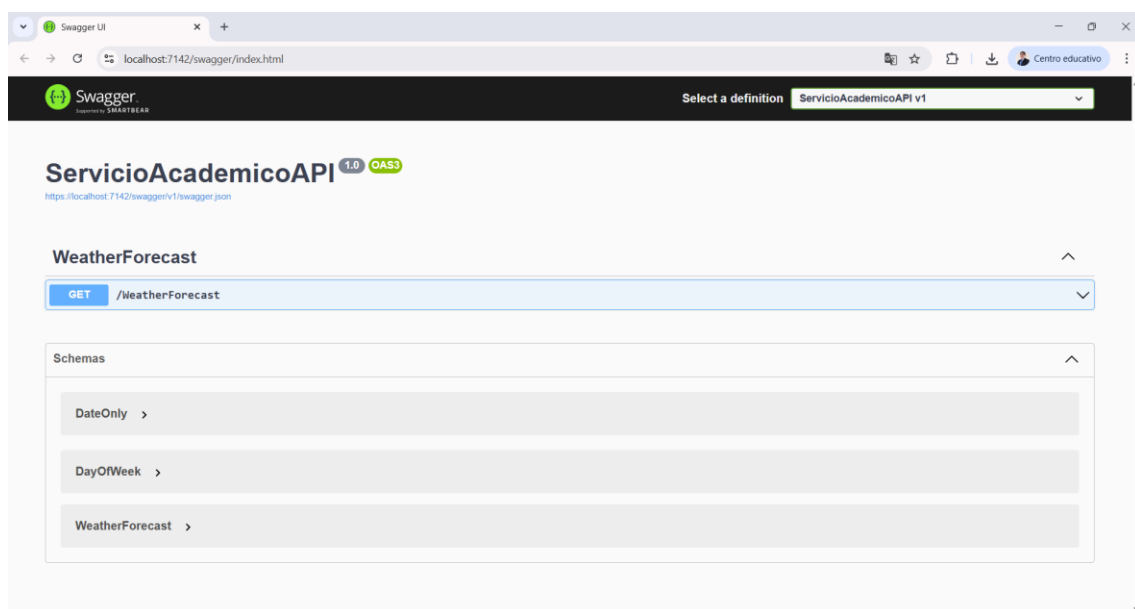
Authentication de campo ⓘ
Ninguno

☒ Configurar para HTTPS ⓘ
☐ Habilitar Docker ⓘ

Sistema operativo de Docker ⓘ
Linux

☒ Usar controladores (desactivar para usar API mínimas) ⓘ
☒ Habilitar compatibilidad con OpenAPI ⓘ
☐ No usar instrucciones de nivel superior ⓘ

6. Crear el proyecto.
7. Ejecutar el proyecto.



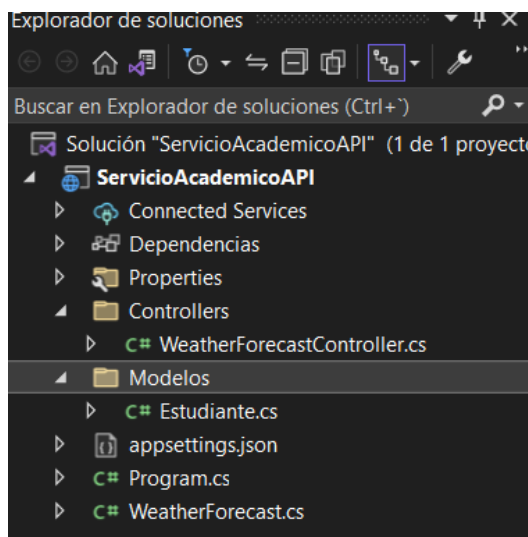
6.2. ACTIVIDAD 02: Creación del Modelo Estudiante

Objetivo: Representar una entidad del sistema dentro de la API.

Procedimiento detallado:

1. Clic derecho en el proyecto → Agregar → Nueva carpeta → Nombre: Modelos.

- Dentro de la carpeta **Modelos** crear la clase simple **Estudiante.cs**.



- Implementar el siguiente código:

```

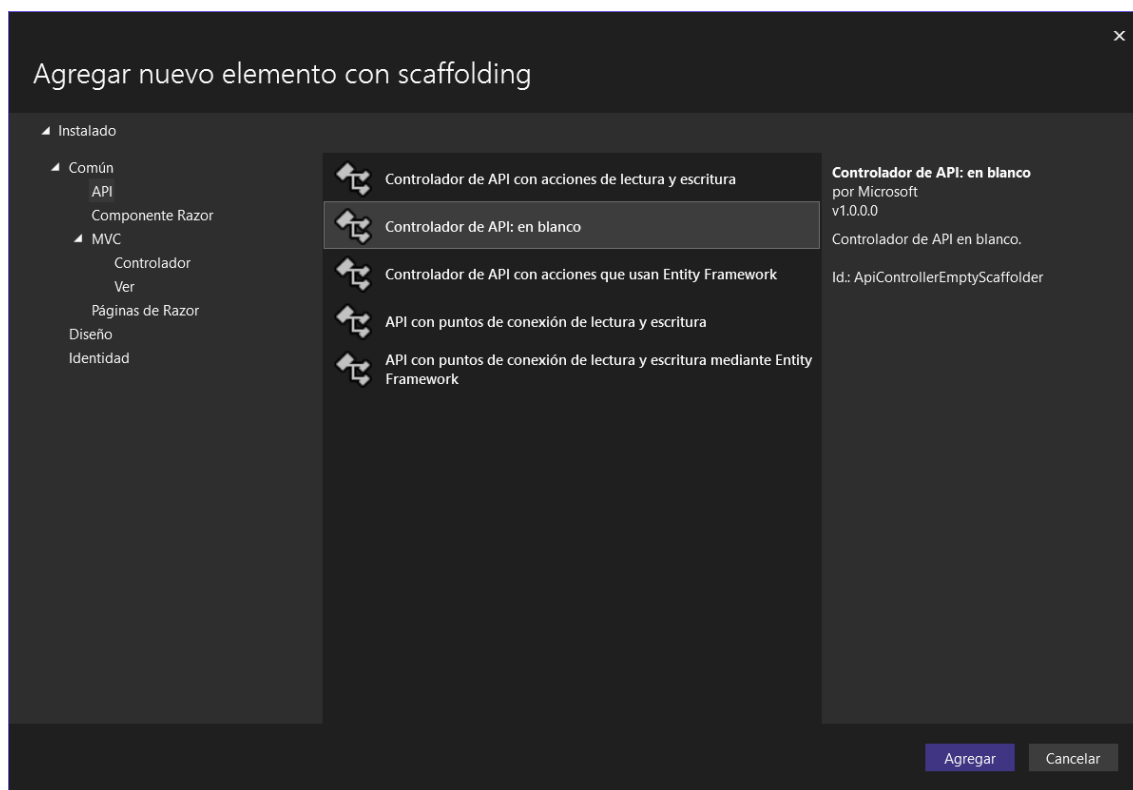
1 namespace ServicioAcademicoAPI.Modelos
2 {
3     0 referencias
4     public class Estudiante
5     {
6         0 referencias
7         public string Codigo { get; set; }
8         0 referencias
9         public string Nombre { get; set; }
10    }
11 }

```

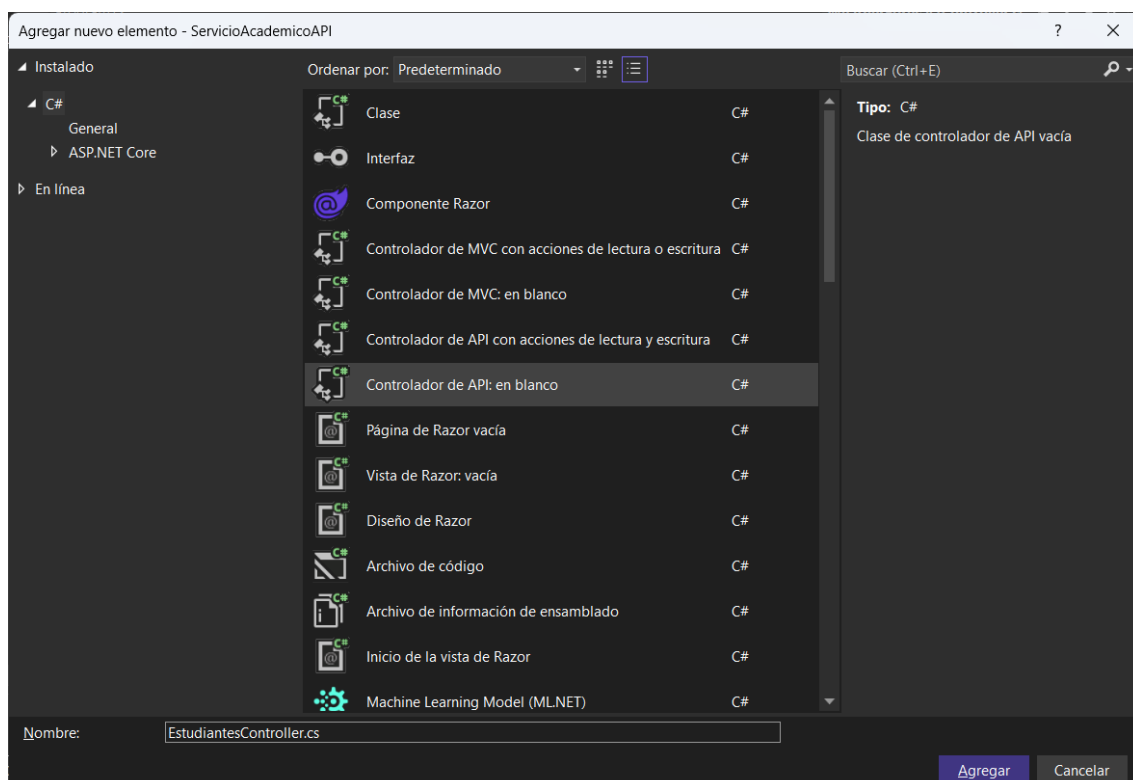
6.3. ACTIVIDAD 3: Implementación de los métodos GET y POST

Objetivo: Implementar los primeros endpoints.

- Clic derecho en la carpeta Controllers → Agregar → Controlador → Controlador API vacío.



2. Nombre: **EstudiantesController**.



3. Implementar el siguiente código base:

```

1  using Microsoft.AspNetCore.Http;
2  using Microsoft.AspNetCore.Mvc;
3  using ServicioAcademicoAPI.Modelos;
4
5  namespace ServicioAcademicoAPI.Controllers
6  {
7      [Route("api/[controller]")]
8      [ApiController]
9      // 0 referencias
10     public class EstudiantesController : ControllerBase
11     {
12         // Lista estática que simula una base de datos temporal
13         private static List<Estudiante> estudiantes = new List<Estudiante>
14         {
15             new Estudiante {Codigo = "E001", Nombre = "Juan Perez" },
16             new Estudiante {Codigo = "E002", Nombre = "María Martinez" }
17         };
18
19         // Método GET: devuelve todos los estudiantes
20         [HttpGet]
21         // 0 referencias
22         public IActionResult Get()
23         {
24             return Ok(estudiantes);
25         }
26
27         // Método POST: agrega un nuevo estudiante
28         [HttpPost]
29         // 0 referencias
30         public IActionResult Post(Estudiante nuevo)
31         {
32             estudiantes.Add(nuevo);
33             return Ok(estudiantes);
34         }
35     }
36 }

```

4. Hacer las pruebas respectivas y realizar las capturas de pantalla respectivas.

Explicación del código:

1. **[ApiController]** Indica que esta clase funcionará como controlador de API. Habilita validaciones automáticas y manejo de respuestas HTTP.
2. **[Route("api/[controller]")]** Define la ruta base del controlador.

En este caso, la ruta será:

```
https://localhost:puerto/api/estudiantes
```

El nombre *estudiantes* proviene automáticamente del nombre del controlador.

3. **Lista estática estudiantes** Simula una base de datos en memoria. No es persistente: si se detiene la aplicación, la información se pierde.

4. Método GET

```
[HttpGet]
public IActionResult Get()
```

- Responde a solicitudes HTTP GET.
- Devuelve todos los estudiantes.
- **Ok(estudiantes)** genera:
 - Código HTTP 200
 - Respuesta en formato JSON

5. Método POST

```
[HttpPost]
public IActionResult Post(Estudiante nuevo)
```

- Recibe datos en el cuerpo de la solicitud.
- ASP.NET Core convierte automáticamente el JSON recibido en un objeto Estudiante.
- Agrega el nuevo estudiante a la lista.
- Devuelve la lista actualizada.

6.4. ACTIVIDAD 4: Implementación de GET por código, PUT y DELETE

Objetivo: Completar operaciones CRUD.

1. Agregar dentro del mismo controlador:

```
[HttpGet("{codigo}")]
0 referencias
public IActionResult GetPorCodigo(string codigo)
{
    var estudiante = estudiantes.FirstOrDefault(e => e.Codigo == codigo);
    if (estudiante == null)
        return NotFound("Estudiante no encontrado");

    return Ok(estudiante);
}

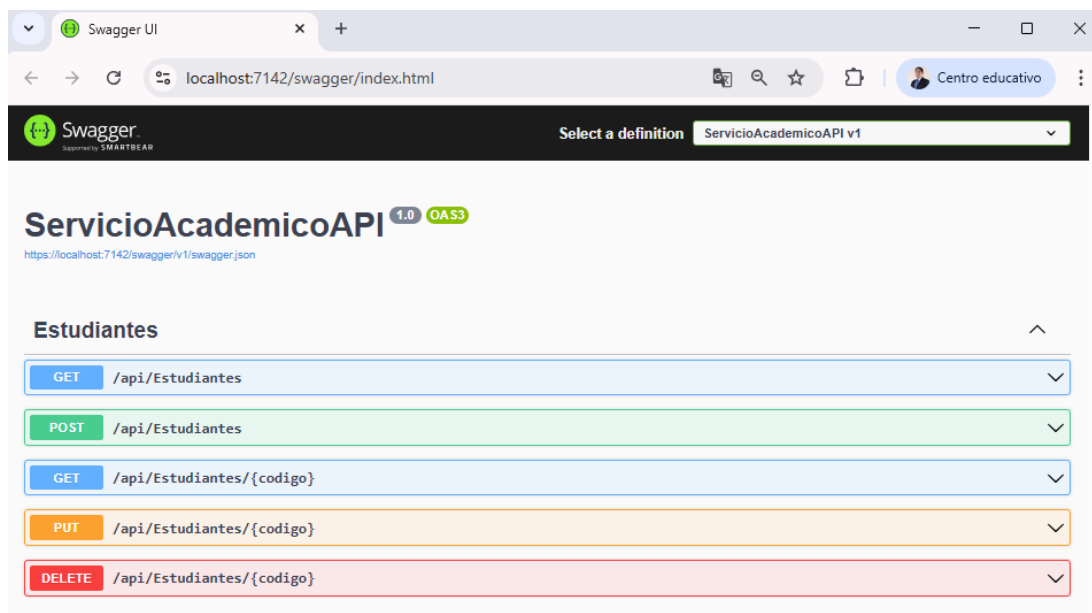
[HttpPut("{codigo}")]
0 referencias
public IActionResult Put(string codigo, Estudiante actualizado)
{
    var estudiante = estudiantes.FirstOrDefault(e => e.Codigo == codigo);
    if (estudiante == null)
        return NotFound();

    estudiante.Nombre = actualizado.Nombre;
    return Ok(estudiante);
}

[HttpDelete("{codigo}")]
0 referencias
public IActionResult Delete(string codigo)
{
    var estudiante = estudiantes.FirstOrDefault(e => e.Codigo == codigo);
    if (estudiante == null)
        return NotFound();

    estudiantes.Remove(estudiante);
    return Ok(estudiantes);
}
```

2. Ejecutar el proyecto:



3. Probar cada **endpoint** y obtener las capturas de pantallas para cada uno de los 5 servicios.
4. Verificar códigos de respuesta HTTP.

7. EJERCICIOS PROPUESTOS

7.1. EJERCICIO 1: Consumo de una Web API desde una aplicación de consola

Enunciado: En una arquitectura cliente-servidor, las aplicaciones cliente consumen servicios expuestos por una Web API. En este ejercicio, el estudiante deberá implementar una aplicación de consola que actúe como cliente del servicio **ServicioAcademicoAPI**, desarrollado previamente.

Objetivo del ejercicio: Consumir una Web API REST desde una aplicación de consola utilizando **HttpClient**, comprendiendo el flujo de comunicación entre aplicaciones.

Indicaciones:

1. Ejecutar el proyecto **ServicioAcademicoAPI** y verificar que se encuentre activo.
2. Abrir el proyecto de consola **ModuloEstudiantes**.
3. Modificar el método Main para que sea asíncrono.
4. Implementar una solicitud HTTP GET hacia el endpoint:

```
GET /api/estudiantes
```

5. Mostrar en consola:
 - Un mensaje indicando que se está enviando la solicitud.
 - El código de respuesta HTTP recibido.
 - El contenido de la respuesta en formato JSON.
6. Mantener la ventana de consola abierta hasta que el usuario presione una tecla.

Resultado esperado:

- La aplicación de consola muestra correctamente la lista de estudiantes obtenida desde la Web API.
- Se evidencia la comunicación correcta entre el cliente (consola) y el servidor (API REST).

7.2. EJERCICIO 2: Validación y mejora del Servicio

Objetivo del ejercicio: Mejorar la robustez de la API.

Indicaciones:

- Implementar validaciones para que el nombre no esté vacío.
- Retornar códigos de estado adecuados (400, 404).
- Agregar mensajes personalizados en las respuestas.

Resultado esperado:

- API con validaciones básicas y mejor manejo de errores.

7.3. EJERCICIO 3: Integración avanzada desde Consola

Objetivo del ejercicio: Simular un cliente más completo.

Indicaciones:

- Implementar menú interactivo que permita:
 - Listar estudiantes
 - Buscar por código
 - Registrar nuevo estudiante
 - Eliminar estudiante

Resultado esperado:

- Aplicación de consola funcionando como cliente completo de la API.

8. PROCESO DE EVALUACIÓN:

Criterios de evaluación de la guía:

Criterios de evaluación	Descripción	Puntaje por ítem
1. Actividades de la Guía	Desarrollo de los ejercicios de la Guía	05
2. Ejercicio Propuesto 01	Desarrollo del ejercicio propuesto 01	05
3. Ejercicio Propuesto 02	Desarrollo del ejercicio propuesto 02	05
4. Ejercicio Propuesto 03	Desarrollo del ejercicio propuesto 03	05
TOTAL		20